

FEDERAL UNIVERSITY OF SCIENCE AND TECHNOLOGY OF MINAS GERAIS  
SCIENTIFIC INITIATION REPORT



# COMPARISON OF THE IMPLEMENTATION OF NUMERICAL METHODS ACROSS DIFFERENT PROGRAMMING PARADIGMS

Autor

Enzo Rocha Leite Diniz Ribas

Orientador

Luis Alberto D'Afonseca

BELO HORIZONTE  
27 DE SETEMBRO DE 2026

# Resumo

---

Esse texto é um exemplo do uso da classe L<sup>A</sup>T<sub>E</sub>X que define a formatação das dissertações de Mestrado do PROFMAT no CEFET-MG.

Palavras-chave: Matemática; Ensino; Ciência.

# Abstract

---

This text is an example of the use of the  $\text{\LaTeX}$  class that defines the formatting of Master's dissertations of PROFMAT at CEFET-MG.

Keywords: Mathematics; Teaching; Science.

# Sumário

---

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>5</b>  |
| <b>2</b> | <b>Literature Review</b>                                         | <b>10</b> |
| 2.1      | Numerical Methods . . . . .                                      | 10        |
| 2.2      | Errors in Numerical Methods . . . . .                            | 11        |
| 2.3      | Numerical Representation and Floating-Point Arithmetic . . . . . | 12        |
| 2.4      | Programming Paradigms . . . . .                                  | 13        |
| 2.5      | Computational Comparison Between Programming Paradigms . . . . . | 14        |
| <b>3</b> | <b>Scientific and Academic Productions</b>                       | <b>17</b> |
| 3.1      | Scientific Papers and Manuscripts . . . . .                      | 17        |
| 3.2      | Conference Contributions and Posters . . . . .                   | 17        |
| 3.3      | Books and Book Chapters . . . . .                                | 18        |
| 3.4      | Presentations and Other Productions . . . . .                    | 18        |
|          | <b>Referências</b>                                               | <b>19</b> |
|          | <b>Apêndices</b>                                                 |           |
| <b>A</b> | <b>Projeto e Planos de Trabalho - Enzo Ribas</b>                 | <b>21</b> |
| A.1      | Projeto . . . . .                                                | 21        |
| A.2      | Plano de Trabalho . . . . .                                      | 21        |
| A.2.1    | Introdução/Justificativa . . . . .                               | 21        |
| A.2.2    | Objetivos . . . . .                                              | 22        |
| A.2.3    | Método Científico . . . . .                                      | 22        |
| A.2.4    | Habilidades Adquiridas . . . . .                                 | 23        |
| A.2.5    | Referências . . . . .                                            | 23        |
| <b>B</b> | <b>Propostas para Novos Projetos</b>                             | <b>28</b> |
| B.1      | Proposta 1 . . . . .                                             | 28        |
| B.1.1    | Resumo . . . . .                                                 | 28        |
| B.1.2    | Introdução/Justificativa . . . . .                               | 28        |
| B.1.3    | Objetivos . . . . .                                              | 28        |
| B.1.4    | Metodologia . . . . .                                            | 28        |
| B.1.5    | Referências . . . . .                                            | 28        |
| B.1.6    | Cronograma . . . . .                                             | 28        |

# 1 Introduction

---

Numerical methods constitute a fundamental area of Applied Mathematics and Scientific Computing, providing techniques for obtaining approximate solutions to mathematical problems for which analytical solutions are unavailable, unknown, or computationally impractical. Problems involving roots of nonlinear equations, systems of equations, numerical integration, interpolation, and differential equations are commonly encountered in engineering, physics, computer science, and mathematical modeling [1, 2].

Although numerical methods are defined through mathematical formulations, their practical application depends directly on computational implementation. Consequently, several computational aspects can influence their behavior, including the programming language, programming paradigm, control structures, numerical representation, compiler or interpreter, and the limitations inherent to finite-precision arithmetic [3].

The same numerical method may therefore assume significantly different computational structures depending on the programming paradigm adopted. Programming paradigms provide different mechanisms for expressing computation, state, control flow, and abstraction. In imperative programming, algorithms are typically described as sequences of instructions that explicitly modify the state of a program. Mutable variables, conditional statements, and iterative control structures are therefore natural mechanisms for representing iterative numerical procedures [4, 5].

Functional programming, in contrast, emphasizes the evaluation and composition of functions rather than the explicit modification of program state. Concepts such as immutability, recursion, higher-order functions, and function composition provide alternative ways of expressing computational processes. Functional programming has also been associated with modularity and with the ability to decompose programs through higher-order functions and lazy evaluation [6].

This distinction becomes particularly relevant when implementing iterative numerical algorithms. Consider, for example, an iterative sequence defined by

$$x_{n+1} = g(x_n). \tag{1.1}$$

In an imperative implementation, this process can naturally be represented by repeatedly updating a variable inside a loop. In a functional implementation, the same recurrence can be expressed through successive function applications, recursive definitions, or transformations over immutable values.

Although these representations describe the same mathematical recurrence, their computational implementations can differ in terms of code structure, expressiveness,

modularity, memory behavior, and execution time. Therefore, the choice of programming paradigm is not merely a syntactic decision; it also determines how the mathematical algorithm is represented and executed by the computer.

This problem is particularly relevant for iterative numerical methods, in which the same operation may be performed hundreds, thousands, or millions of times. In such cases, differences introduced by the programming language, runtime system, compiler optimizations, memory-management strategy, or evaluation model may significantly affect the final computational cost.

Another fundamental issue arises from the fact that digital computers cannot, in general, represent arbitrary real numbers exactly. Instead, real-valued quantities are represented using finite sets of machine numbers, most commonly through floating-point arithmetic [3, 7].

The IEEE 754 standard defines widely adopted formats and operations for binary and decimal floating-point arithmetic, including rules for rounding, exceptional values, and arithmetic behavior [7]. Because the number of available bits is finite, many real numbers cannot be represented exactly and must instead be approximated.

For example, a real number  $x$  that possesses a finite decimal representation may have an infinite binary expansion. When represented using a finite number of bits, its stored value can be written conceptually as

$$\tilde{x} = x + \delta, \tag{1.2}$$

where  $\delta$  denotes the error introduced by finite-precision representation.

Such errors are especially important in iterative numerical algorithms because results produced at one iteration are commonly used as inputs to subsequent iterations. As a result, rounding errors can propagate, accumulate, or be amplified depending on the numerical properties of the algorithm [3].

The research presented in this work is therefore situated at the intersection of three principal topics: numerical methods, programming paradigms, and floating-point arithmetic.

The central research question can be formulated as follows:

*How do different programming paradigms and programming languages influence the implementation, computational performance, readability, and numerical behavior of algorithms used in numerical methods?*

To investigate this question, implementations in C, Python, and Haskell are considered. C provides a predominantly imperative programming model, offering explicit control over variables, control structures, data representation, and memory organization [5].

Python is a general-purpose programming language widely used in scientific computing. Its ecosystem has become particularly important for numerical and array-based computation, notably through libraries such as NumPy [8]. Although Python supports multiple programming styles, numerical algorithms can naturally be expressed using conventional imperative constructs such as loops, assignments, and conditional statements.

Haskell, in contrast, is a purely functional programming language characterized by immutable values, first-class functions, higher-order functions, recursion, strong static typing, and non-strict evaluation [9, 10]. These characteristics make Haskell especially suitable for investigating how numerical algorithms can be expressed without relying on explicit mutable state.

The use of these three languages allows different representations of the same numerical algorithm to be investigated while exposing distinct computational models.

Among the numerical algorithms considered in this work are iterative methods such as fixed-point iteration and the Newton–Raphson method. Such algorithms are particularly suitable for comparisons between programming paradigms because their mathematical formulations are based on recursively defined sequences of approximations [1, 2].

In fixed-point iteration, for example, the objective is to determine a value  $x^*$  satisfying

$$x^* = g(x^*). \quad (1.3)$$

Starting from an initial approximation  $x_0$ , a sequence is generated according to

$$x_{n+1} = g(x_n), \quad (1.4)$$

until a prescribed convergence criterion is satisfied.

The convergence of fixed-point iteration depends on mathematical properties of the function  $g$  and on the initial approximation. Under appropriate conditions, the generated sequence converges to a fixed point of the function [1].

This recursive structure highlights one of the central differences between the programming paradigms investigated in this work. In an imperative implementation, the current approximation may be explicitly updated during each loop iteration. In a functional implementation, the sequence may instead be described as successive applications of a function without requiring an explicitly mutable variable.

The Newton–Raphson method provides another important example. Given a nonlinear equation

$$f(x) = 0, \quad (1.5)$$

an iterative approximation is obtained using

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (1.6)$$

Under appropriate regularity assumptions and for sufficiently suitable initial approximations, Newton’s method can exhibit rapid local convergence [1, 2].

Despite the mathematical equivalence of implementations based on the same recurrence relations, their computational characteristics may differ. This work does not attempt to establish one programming paradigm as universally superior to another. Instead, it investigates quantitatively and qualitatively how different implementation models behave in the particular context of numerical algorithms.

The quantitative analysis considers metrics such as execution time, memory usage, computational resource consumption, number of iterations, and implementation size. Whenever applicable, compiler optimization levels and language execution models are also taken into account.

Experimental comparisons between programming languages require particular care. Differences in execution time cannot necessarily be attributed to the programming paradigm alone, since performance is also influenced by compiler technology, runtime implementation, optimization strategies, memory management, data representation, and hardware architecture.

For this reason, the experiments presented in this work are conducted under controlled conditions, using the same hardware platform and equivalent testing conditions whenever possible. Different test functions and computational scenarios are employed to examine the behavior of the implementations under varying levels of numerical and computational complexity.

In addition to quantitative metrics, qualitative characteristics are examined, including readability, modularity, expressiveness, maintainability, and the degree of correspondence between the mathematical formulation of an algorithm and its source-code representation.

Functional programming is particularly interesting in this context because higher-order functions and abstraction mechanisms can improve program modularity and permit mathematical operations to be represented compositionally [6]. However, properties such as recursion and lazy evaluation may also result in execution and memory behaviors that differ from those encountered in conventional imperative implementations.

A further component of this research concerns floating-point arithmetic. Numerical algorithms are executed using finite computational representations, which means that mathematical operations over real numbers are replaced by operations over a finite subset of representable values. The consequences of this approximation include rounding error, loss of significance, cancellation, underflow, overflow, and other finite-precision effects [3, 7].

These effects are fundamental when interpreting the experimental results. Differences between implementations must therefore not be analyzed solely in terms of execution speed, but also with respect to the numerical behavior of the algorithms and the arithmetic used by the underlying computational system.

The general objective of this work is to analyze the relationship between programming paradigms and numerical methods, evaluating how different implementation strategies influence both computational and numerical properties of the algorithms.

The specific objectives are:

- to study the mathematical foundations of the numerical methods considered in this work;
- to implement numerical algorithms using C, Python, and Haskell;
- to represent equivalent numerical methods using imperative and functional programming approaches;

- to compare the implementations in terms of execution time, memory consumption, and computational resource usage;
- to analyze differences in readability, modularity, and expressiveness among the implementations;
- to investigate the relationship between recursion, iterative control structures, and computational performance;
- to study the effects of floating-point representation on numerical algorithms;
- to investigate the propagation of representation and rounding errors throughout iterative numerical processes;
- to identify advantages and limitations associated with imperative and functional programming approaches in the implementation of numerical methods.

By combining these perspectives, this research aims to provide an analysis that extends beyond a direct comparison of programming languages. The broader objective is to understand how different computational models interact with the mathematical characteristics of numerical algorithms and how those interactions affect performance, implementation structure, and numerical reliability.

# 2 Literature Review

---

The theoretical foundation of this work involves concepts from different areas of Mathematics and Computer Science. In particular, the literature review is organized around four main topics: numerical methods, error analysis, floating-point arithmetic, and programming paradigms.

These topics are directly related within the context of this research. Numerical methods provide the mathematical algorithms used in the experiments; error analysis provides the theoretical framework required to assess the quality of numerical approximations; floating-point arithmetic establishes the limitations of numerical representation in digital computers; and the study of programming paradigms provides the conceptual basis for analyzing different ways of implementing numerical algorithms [1–3].

## 2.1 Numerical Methods

Numerical analysis studies mathematical and computational techniques designed to obtain approximate solutions to mathematical problems. Such techniques are particularly important when analytical solutions are unavailable, difficult to obtain, or computationally impractical [1, 2].

Classical numerical analysis literature identifies approximation, error, convergence, stability, and computational efficiency as fundamental aspects in the study and implementation of numerical algorithms [1, 3].

One of the classical problems in numerical analysis is the determination of roots of nonlinear equations. Given a function

$$f(x) = 0, \tag{2.1}$$

the objective is to determine a value  $x^*$  that satisfies the equation, or to compute an approximation sufficiently close to an exact root.

Several numerical algorithms may be employed for this purpose, each presenting different convergence properties and computational costs. Among the methods considered in this work are fixed-point iteration and the Newton–Raphson method [1, 2].

Fixed-point iteration begins by transforming the original problem

$$f(x) = 0 \tag{2.2}$$

into an equivalent equation of the form

$$x = g(x). \tag{2.3}$$

Starting from an initial approximation  $x_0$ , a sequence is then constructed according to

$$x_{n+1} = g(x_n). \quad (2.4)$$

If appropriate convergence conditions are satisfied, the sequence  $\{x_n\}$  converges to a fixed point  $x^*$  of the function  $g$ , satisfying

$$g(x^*) = x^*. \quad (2.5)$$

The convergence of fixed-point iteration depends on the properties of the function  $g$ , the interval considered, and the initial approximation. A frequently used sufficient condition involves the contraction properties of  $g$  in a neighborhood containing the fixed point [1].

The recursive nature of this expression makes fixed-point iteration particularly relevant to the present research. Its mathematical formulation can be translated either into explicit iterative structures or into recursive function definitions, thereby allowing a direct comparison between imperative and functional programming approaches.

Another important method is the Newton–Raphson method. Starting from an initial approximation  $x_0$ , successive approximations are computed using

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (2.6)$$

Under appropriate assumptions, including sufficient smoothness of the function and an initial approximation sufficiently close to a simple root, Newton’s method exhibits quadratic local convergence [1, 2].

However, its practical behavior also depends on the function under analysis, the selected initial approximation, and the numerical evaluation of both the function and its derivative.

These methods illustrate a common characteristic of many numerical algorithms: the repeated application of a mathematical procedure until a specified criterion is satisfied. This property makes iterative numerical methods particularly suitable for investigating how different programming models represent repeated computation.

## 2.2 Errors in Numerical Methods

A numerical solution is generally an approximation to an exact mathematical quantity. Consequently, the analysis of errors associated with numerical approximation is a fundamental component of numerical analysis [1, 2].

Let  $x$  denote the exact value of a quantity and  $\tilde{x}$  a numerical approximation. The absolute error may be defined as

$$E_a = |x - \tilde{x}|, \quad (2.7)$$

while the relative error can be expressed as

$$E_r = \frac{|x - \tilde{x}|}{|x|}, \quad (2.8)$$

provided that  $x \neq 0$ .

Relative error is particularly useful because it relates the magnitude of the approximation error to the magnitude of the quantity being approximated, providing a scale-independent measure of accuracy [2].

In many practical numerical problems, however, the exact value is not known. In such cases, stopping criteria based on successive approximations may be used.

A common criterion consists of terminating the iterative procedure when

$$|x_{n+1} - x_n| < \varepsilon, \quad (2.9)$$

where  $\varepsilon$  represents a predetermined tolerance.

The selection of the tolerance directly affects the computational behavior of the algorithm. A relatively large tolerance may result in premature termination, while an excessively small tolerance may increase the number of iterations without necessarily producing a proportional improvement in accuracy.

This limitation becomes especially important in finite-precision computation, where the accuracy obtainable from an algorithm is ultimately constrained by the arithmetic used by the machine [3].

## 2.3 Numerical Representation and Floating-Point Arithmetic

Digital computer systems operate with finite storage and finite numerical representations. Consequently, only a finite subset of the real numbers can be represented exactly by a computer [11, 3].

Numerical representation in digital systems is based primarily on positional number systems, particularly the binary system. In general, a number in base  $b$  may be represented by

$$x = \sum_{i=m}^n d_i b^i, \quad (2.10)$$

where  $d_i$  denotes a digit in the chosen base.

Modern computer systems commonly represent non-integer numerical values using floating-point formats. In simplified form, a floating-point number may be written as

$$x = \pm m \times b^e, \quad (2.11)$$

where  $m$  represents the significand,  $b$  denotes the numerical base, and  $e$  represents the exponent.

The term *significand* is generally more precise than *mantissa* in the context of modern floating-point arithmetic, although both terms are frequently encountered in the literature.

Because only a finite number of bits is available for representing these components, many real numbers cannot be stored exactly. Instead, the computer stores a nearby representable value [11, 3].

This phenomenon is particularly evident for decimal fractions whose expansions do not terminate in binary representation. For example, values such as  $0.1_{10}$  cannot be represented exactly using a finite binary expansion.

Floating-point arithmetic therefore introduces rounding errors as a natural consequence of finite representation. These errors may interact with other numerical phenomena, including loss of significance, cancellation, underflow, overflow, and propagation of rounding error [11, 3].

The IEEE 754 standard defines widely adopted floating-point formats and operations, including rules for binary and decimal arithmetic, rounding, special values such as infinity and NaN, and exceptional numerical conditions [7].

In iterative numerical algorithms, finite-precision effects are particularly important because the result obtained during one iteration frequently becomes the input to the next iteration. Consequently, rounding errors introduced at an early stage may propagate throughout the computation.

Depending on the stability and conditioning of the numerical problem, such errors may remain small, accumulate, or be amplified during successive operations [3].

The study of floating-point representation is therefore essential for the interpretation of the experimental results obtained in this work. Differences between implementations should not be evaluated exclusively in terms of execution time, but also in terms of numerical behavior and finite-precision effects.

## 2.4 Programming Paradigms

A programming paradigm may be understood as a conceptual model for organizing and expressing computation. Different paradigms provide distinct abstractions for representing data, state, operations, and control flow [4].

The two principal paradigms considered in this work are imperative programming and functional programming.

In the imperative paradigm, a program can be understood as a sequence of instructions that modify computational state. Assignments, loops, and conditional structures explicitly determine how the state of the program changes during execution [4, 5].

For example, a numerical recurrence

$$x_{n+1} = g(x_n) \tag{2.12}$$

can be implemented by storing the current approximation in a variable and successively updating its value during a loop.

This model is closely related to the conventional state-oriented execution model of modern processors and is naturally expressed in languages such as C. Python also supports extensive use of imperative constructs, although it is a multiparadigm language and provides additional abstractions.

Functional programming, in contrast, emphasizes function evaluation and composition rather than explicit state modification. Common characteristics include immutable values, first-class functions, higher-order functions, function composition, and recursive definitions [6, 10].

In a functional formulation, the same numerical recurrence may be expressed through repeated applications of a function:

$$x_1 = g(x_0), \tag{2.13}$$

$$x_2 = g(g(x_0)), \tag{2.14}$$

$$x_3 = g(g(g(x_0))), \tag{2.15}$$

and, more generally,

$$x_n = g^{(n)}(x_0). \tag{2.16}$$

This representation has a direct relationship with the mathematical formulation of iterative methods, since computation may be expressed as successive applications and compositions of functions rather than explicit modifications of mutable state.

Functional programming also enables the use of higher-order functions, in which functions may receive other functions as arguments or return them as results. Hughes argues that higher-order functions and lazy evaluation can serve as important mechanisms for improving modularity and decomposing programs into reusable components [6].

Haskell is one of the principal languages associated with purely functional programming. The Haskell language is non-strict and purely functional, and functions are first-class values. Its evaluation model allows expressions to remain unevaluated until their values are required [9, 10]. The official Haskell 2010 Language Report defines the language and its non-strict semantics [9].

These properties enable algorithms to be expressed using abstractions that differ substantially from conventional imperative implementations. At the same time, lazy evaluation and immutable data structures may result in execution and memory-access patterns that differ from those of strictly evaluated imperative programs.

## 2.5 Computational Comparison Between Programming Paradigms

The comparison between programming paradigms cannot be reduced solely to the execution time of a program. A meaningful analysis must consider multiple dimensions of software and computational behavior.

In this work, the comparison includes both quantitative and qualitative criteria.

The quantitative metrics considered include:

- execution time;
- memory usage;

- computational resource usage;
- number of iterations;
- implementation size;
- influence of compiler optimization.

The qualitative characteristics considered include:

- readability;
- modularity;
- expressiveness;
- maintainability;
- correspondence between the mathematical formulation and its computational implementation.

The comparison must also account for the fact that C, Python, and Haskell possess substantially different execution models.

C is generally compiled ahead of time into machine code and provides relatively direct control over memory organization, data representation, and low-level operations [5].

Python, in its most widely used implementation, CPython, compiles source code into bytecode executed by the Python virtual machine. At the same time, performance-critical scientific operations are frequently delegated to native compiled libraries. This distinction is especially relevant in numerical computing, where libraries such as NumPy provide optimized array operations implemented largely outside ordinary Python execution [8].

Haskell implementations such as the Glasgow Haskell Compiler typically compile programs to native code while preserving semantic properties associated with pure functional programming and non-strict evaluation. Consequently, the runtime behavior of Haskell differs substantially from that of both C and Python [9].

Experimental differences among implementations therefore cannot be attributed exclusively to the programming paradigm. Observed performance results emerge from the interaction among the numerical algorithm, programming paradigm, programming language, compiler or interpreter, runtime system, optimization strategies, memory representation, and hardware architecture.

For this reason, the experiments conducted in this research attempt to keep external conditions constant whenever possible and explicitly document the tools, language versions, compiler versions, compiler optimization options, and hardware characteristics used during benchmarking.

Such methodological control is necessary to reduce confounding factors and to avoid generalizing language-specific performance results into broader claims about entire programming paradigms.

The objective of the experimental comparison is therefore not to determine an absolute superiority between imperative and functional programming. Instead, the purpose is to identify how their different computational models interact with the mathematical and iterative properties of numerical algorithms.

This approach allows performance, memory behavior, implementation structure,

and numerical characteristics to be analyzed together, providing a broader assessment of the consequences of programming-paradigm selection in scientific and numerical computing.

## 3 Scientific and Academic Productions

---

This chapter presents the main scientific and academic productions developed within the scope of the research project, including scientific papers, conference contributions, posters, books, book chapters, and other academic activities related to the research.

### 3.1 Scientific Papers and Manuscripts

#### **O Computador Sempre Erra: Uma Exploração Gráfica dos Efeitos da Aritmética de Ponto Flutuante**

**Authors:** Ribas et al.

**Status:** Manuscript in development.

### 3.2 Conference Contributions and Posters

#### **Análise Visual de Erros de Ponto Flutuante e das Units of Last Place em Experimentos Computacionais**

**Authors:** Enzo Rocha Leite Diniz Ribas, L. A. D'Afonseca, and L. M. Rocha.

**Event:** XLV Congresso Nacional de Matemática Aplicada e Computacional (CN-MAC 2026).

#### **Programação Funcional: do Cálculo Lambda à Implementação de Métodos Numéricos**

**Authors:** E. R. L. D. Ribas, L. A. D'Afonseca, and L. M. Rocha.

**Event:** 22nd Science & Technology Week of Federal University of Science and Technology of Minas Gerais (UFCIMG). (2026/2)

#### **Explorando alguns efeitos dos erros de Ponto Flutuante**

**Authors:** E. R. L. D. Ribas, D. S. Pazini, L. A. D'Afonseca, and L. M. Rocha.

**Event:** I Workshop Mineiro de Matemática Aplicada. (WWMA 2026/1)

### **3.3 Books and Book Chapters**

#### **Numerical Methods Implementation with Functional Programming**

**Authors:** Enzo Rocha Leite Diniz Ribas, L. A. D'Afonseca, and L. M. Rocha.

### **3.4 Presentations and Other Productions**

This section is reserved for oral presentations, invited talks, academic presentations, and other research-related productions.

# Referências

---

- 1 BURDEN, R. L.; FAIRES, J. D.; BURDEN, A. M. *Numerical Analysis*. 10. ed.: Cengage Learning, 2016. ISBN 9781305253667.
- 2 CHAPRA, S. C.; CANALE, R. P. *Numerical Methods for Engineers*. 8. ed.: McGraw-Hill Education, 2021. ISBN 9781260232073.
- 3 HIGHAM, N. J. *Accuracy and Stability of Numerical Algorithms*. 2. ed. Philadelphia, PA: Society for Industrial e Applied Mathematics, 2002. ISBN 9780898715217. DOI: [10.1137/1.9780898718027](https://doi.org/10.1137/1.9780898718027).
- 4 SCOTT, M. L. *Programming Language Pragmatics*. 4. ed.: Morgan Kaufmann, 2015.
- 5 KERNIGHAN, B. W.; RITCHIE, D. M. *The C Programming Language*. 2. ed.: Prentice Hall, 1988. ISBN 9780131103627.
- 6 HUGHES, J. Why Functional Programming Matters. *The Computer Journal*, v. 32, n. 2, p. 98–107, 1989. DOI: [10.1093/comjnl/32.2.98](https://doi.org/10.1093/comjnl/32.2.98).
- 7 IEEE. *IEEE Standard for Floating-Point Arithmetic*. 2019. DOI: [10.1109/IEEESTD.2019.8766229](https://doi.org/10.1109/IEEESTD.2019.8766229).
- 8 HARRIS, C. R. *et al.* Array Programming with NumPy. *Nature*, v. 585, n. 7825, p. 357–362, 2020. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- 9 MARLOW, S. (ed.). *Haskell 2010 Language Report*. 2010. Disponível em: <https://www.haskell.org/onlinereport/haskell2010/>.
- 10 HUTTON, G. *Programming in Haskell*. 2. ed.: Cambridge University Press, 2016.
- 11 GOLDBERG, D. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, Association for Computing Machinery, v. 23, n. 1, p. 5–48, mar. 1991. DOI: [10.1145/103162.103163](https://doi.org/10.1145/103162.103163).

“`\latex`

# Apêndice A

## Projeto e Planos de Trabalho - Enzo Ribas

---

### A.1 Projeto

Dados do projeto de pesquisa

**Código:** PIC704-2024

**Título:** Comparação da Implementação de Métodos Numéricos em Diferentes Paradigmas de Programação

**Orientador:** Luis Alberto D'Afonseca

**Co-orientador:** Lucas Martins Rocha

**Discente:** Enzo Rocha Leite Diniz Ribas

**Tipo de Bolsa:** Voluntário (IC)

**Direcionamento:** Iniciação Científica

**Status:** Semana C&T Pendente

**Editais:** Edital DPPG nº 73/2024 – PIBIC FAPEMIG

**Cota:** PIBIC FAPEMIG 2025–2026 (28/01/2025 a 28/02/2026)

Área de Conhecimento

**Grande Área:** Ciências Exatas e da Terra

**Área:** Matemática

**Subárea:** Matemática Aplicada

**Especialidade:** Análise Numérica

### A.2 Plano de Trabalho

**Título:** Implementação de métodos numéricos em linguagens funcionais

#### A.2.1 Introdução/Justificativa

Os métodos numéricos são ferramentas essenciais nas áreas de engenharia e ciências, onde desempenham um papel fundamental na modelagem de sistemas complexos. Textos e publicações especializadas destacam a importância dessas técnicas no contexto computacional, como no livro *Applied Numerical Methods with MATLAB for Engineers and Scientists*, de Steven Chapra, que enfatiza aplicações práticas. Clássicos como *Numerical Methods for Scientists and Engineers*, de Richard Hamming (1962), estabeleceram fundamentos que

permanecem relevantes, e *Numerical Analysis*, de R. L. Burden e J. D. Faires, continua sendo uma referência atual, abrangendo métodos clássicos e contemporâneos da análise numérica.

Os paradigmas de programação, por sua vez, oferecem diferentes abordagens para a construção de software, influenciando diretamente como métodos numéricos são implementados e executados. Cada paradigma – seja o imperativo, orientado a objetos ou funcional – apresenta suas particularidades em termos de estrutura de código, mutabilidade de variáveis e organização dos processos computacionais. A escolha do paradigma afeta aspectos como legibilidade, desempenho e facilidade de manutenção do código, aspectos cruciais para o desenvolvimento de soluções numéricas eficientes.

### A.2.2 Objetivos

O objetivo deste projeto é investigar e comparar a implementação de métodos numéricos em diferentes paradigmas de programação, analisando como cada abordagem influencia aspectos essenciais do desenvolvimento e execução desses métodos. Para isso, os métodos selecionados serão implementados em linguagens representativas dos paradigmas imperativo, orientado a objetos e funcional.

Através dessa análise, o projeto busca identificar as vantagens e desvantagens de cada paradigma em termos de eficiência, legibilidade, facilidade de manutenção e expansão do código, fornecendo uma visão mais clara sobre o impacto das escolhas de paradigma na resolução de problemas numéricos.

Pretendemos que, ao realizar este projeto, os alunos adquiram uma compreensão prática e teórica dos principais paradigmas de programação e sua aplicabilidade em problemas científicos, desenvolvendo habilidades de programação em diversas linguagens. Eles aprenderão sobre a implementação dos métodos numéricos fundamentais, além de técnicas de análise de desempenho e complexidade de código.

O projeto permite que explorem os impactos de diferentes paradigmas em termos de eficiência, legibilidade e manutenção, promovendo uma visão crítica sobre a escolha de linguagem e paradigma em cenários de desenvolvimento, especialmente no contexto de aplicações numéricas e científicas.

### A.2.3 Método Científico

A metodologia do projeto envolve a escolha e implementação de métodos numéricos, como o método de Euler para equações diferenciais ordinárias e o método de Newton–Raphson para raízes de funções, em diferentes paradigmas de programação, com o intuito de comparar suas características.

Para isso, serão selecionadas linguagens representativas dos paradigmas imperativo (C, Octave), orientado a objetos (Java ou Python) e funcional (Haskell ou F#). Cada método será implementado em cada linguagem respeitando as boas práticas e os princípios do respectivo paradigma, assegurando uma comparação coerente e representativa.

Em seguida, as implementações serão avaliadas quanto ao desempenho (tempo de execução e uso de memória), complexidade do código (linhas de código e estrutura

algorítmica) e legibilidade e manutenção (modularidade e clareza). Com esses dados, será feita uma análise comparativa para identificar as vantagens e desvantagens de cada paradigma na implementação de métodos numéricos.

### A.2.4 Habilidades Adquiridas

Este projeto visa proporcionar ao estudante uma formação sólida em métodos numéricos e modelagem matemática, capacitando-o a seguir com pesquisa de nível de mestrado. Os conhecimentos matemáticos adquiridos serão essenciais para o desenvolvimento do projeto e contribuirão significativamente para a compreensão de conteúdos abordados nas disciplinas matemáticas dos cursos de engenharia, fortalecendo a base acadêmica do aluno.

De forma mais específica, ao realizar este projeto, o aluno desenvolverá uma compreensão prática e teórica dos principais paradigmas de programação e sua aplicação em problemas científicos. Ele adquirirá habilidades em diferentes linguagens de programação e aprenderá a implementar métodos numéricos fundamentais, além de dominar técnicas de análise de desempenho, complexidade de código e otimização de soluções.

Esse processo proporcionará ao estudante um entendimento mais profundo sobre como a escolha do paradigma de programação influencia a eficiência e a manutenção de soluções numéricas em contextos reais.

Em particular, este plano de trabalho enfatiza as linguagens funcionais. Essas linguagens são pouco utilizadas, apesar de, em princípio, oferecerem vantagens sobre as procedurais. Neste projeto, pretendemos explorar essas características e avaliar se as vantagens são significativas.

### A.2.5 Referências

1. CHAPRA, S. C.; CANALE, R. P. *Applied Numerical Methods with MATLAB for Engineers and Scientists*. McGraw-Hill Education, 2015.
2. BURDEN, R. L.; FAIRES, J. D. *Numerical Analysis*. Brooks Cole, 2010.
3. GABBRIELLI, M.; MARTINI, S.; GIALLOrenzo, S. *Programming Languages: Principles and Paradigms*. Germany: Springer International Publishing, 2023.
4. MELO, A. C. V. d.; SILVA, F. S. C. d. *Princípios de linguagens de programação*. Brazil: Editora Blucher, 2003.
5. STRUIK, D. J. *A Source Book in Mathematics, 1200–1800*. Princeton University Press, 1987.
6. HAMMING, R. W. *Numerical Methods for Scientists and Engineers*. Dover Publications, 1962.
7. RUGGIERO, M. A. G.; DA ROCHA LOPES, V. L. *Cálculo numérico: aspectos teóricos e computacionais*. Pearson/Makron, 1998.

## Corpo do Plano de Trabalho

### Título

Implementação de métodos numéricos em linguagens funcionais

## Introdução e Justificativa

Os métodos numéricos são ferramentas essenciais nas áreas de engenharia e ciências, onde desempenham um papel fundamental na modelagem de sistemas complexos. Textos e publicações especializadas destacam a importância dessas técnicas no contexto computacional, como no livro *Applied Numerical Methods with MATLAB for Engineers and Scientists*, de Steven Chapra, que enfatiza aplicações práticas. Clássicos como *Numerical Methods for Scientists and Engineers*, de Richard Hamming (1962), estabeleceram fundamentos que permanecem relevantes, e *Numerical Analysis*, de R. L. Burden e J. D. Faires, continua sendo uma referência atual, abrangendo métodos clássicos e contemporâneos da análise numérica.

Os paradigmas de programação, por sua vez, oferecem diferentes abordagens para a construção de software, influenciando diretamente como métodos numéricos são implementados e executados. Cada paradigma – seja o imperativo, orientado a objetos ou funcional – apresenta suas particularidades em termos de estrutura de código, mutabilidade de variáveis e organização dos processos computacionais. A escolha do paradigma afeta aspectos como legibilidade, desempenho e facilidade de manutenção do código, aspectos cruciais para o desenvolvimento de soluções numéricas eficientes.

## Objetivos

O objetivo deste projeto é investigar e comparar a implementação de métodos numéricos em diferentes paradigmas de programação, analisando como cada abordagem influencia aspectos essenciais do desenvolvimento e execução desses métodos.

Para isso, os métodos selecionados serão implementados em linguagens representativas dos paradigmas imperativo, orientado a objetos e funcional. Através dessa análise, o projeto busca identificar as vantagens e desvantagens de cada paradigma em termos de eficiência, legibilidade, facilidade de manutenção e expansão do código, fornecendo uma visão mais clara sobre o impacto das escolhas de paradigma na resolução de problemas numéricos.

Pretendemos que, ao realizar este projeto, os alunos adquiram uma compreensão prática e teórica dos principais paradigmas de programação e sua aplicabilidade em problemas científicos, desenvolvendo habilidades de programação em diversas linguagens. Eles aprenderão sobre a implementação dos métodos numéricos fundamentais, além de técnicas de análise de desempenho e complexidade de código.

O projeto permite que explorem os impactos de diferentes paradigmas em termos de eficiência, legibilidade e manutenção, promovendo uma visão crítica sobre a escolha de linguagem e paradigma em cenários de desenvolvimento, especialmente no contexto de aplicações numéricas e científicas.

## Metodologia

A metodologia do projeto envolve a escolha e implementação de métodos numéricos, como o método de Euler para EDOs e o método de Newton–Raphson para raízes de funções, em diferentes paradigmas de programação, com o intuito de comparar suas características.

Para isso, serão selecionadas linguagens representativas dos paradigmas imperativo (C, Octave), orientado a objetos (Java ou Python) e funcional (Haskell ou F#). Cada método será implementado em cada linguagem respeitando as boas práticas e os princípios do respectivo paradigma, assegurando uma comparação coerente e representativa.

Em seguida, as implementações serão avaliadas quanto ao desempenho (tempo de execução e uso de memória), complexidade do código (linhas de código e estrutura algorítmica) e legibilidade e manutenção (modularidade e clareza).

Com esses dados, será feita uma análise comparativa para identificar as vantagens e desvantagens de cada paradigma na implementação de métodos numéricos.

## Habilidades Adquiridas

Este projeto visa proporcionar ao estudante uma formação sólida em métodos numéricos e modelagem matemática, capacitando-o a seguir com pesquisa de nível de mestrado. Os conhecimentos matemáticos adquiridos serão essenciais para o desenvolvimento do projeto e contribuirão significativamente para a compreensão de conteúdos abordados nas disciplinas matemáticas dos cursos de engenharia, fortalecendo a base acadêmica do aluno.

De forma mais específica, ao realizar este projeto, o aluno desenvolverá uma compreensão prática e teórica dos principais paradigmas de programação e sua aplicação em problemas científicos. Ele adquirirá habilidades em diferentes linguagens de programação e aprenderá a implementar métodos numéricos fundamentais, além de dominar técnicas de análise de desempenho, complexidade de código e otimização de soluções.

Esse processo proporcionará ao estudante um entendimento mais profundo sobre como a escolha do paradigma de programação influencia a eficiência e a manutenção de soluções numéricas em contextos reais.

Em particular, este plano de trabalho enfatiza as linguagens funcionais. Essas linguagens são pouco utilizadas, apesar de, em princípio, oferecerem vantagens sobre as procedurais. Neste projeto, pretendemos explorar essas características e avaliar se as vantagens são significativas.

## Referências

1. CHAPRA, S. C.; CANALE, R. P. *Applied Numerical Methods with MATLAB for Engineers and Scientists*. McGraw-Hill Education, 2015.
2. BURDEN, R. L.; FAIRES, J. D. *Numerical Analysis*. Brooks Cole, 2010.
3. GABBRIELLI, M.; MARTINI, S.; GIALlorenzo, S. *Programming Languages: Principles and Paradigms*. Germany: Springer International Publishing, 2023.
4. MELO, A. C. V. d.; SILVA, F. S. C. d. *Princípios de linguagens de programação*. Brazil: Editora Blucher, 2003.
5. STRUIK, D. J. *A Source Book in Mathematics, 1200–1800*. Princeton University Press, 1987.
6. HAMMING, R. W. *Numerical Methods for Scientists and Engineers*. Dover Publications, 1962.
7. RUGGIERO, M. A. G.; DA ROCHA LOPES, V. L. *Cálculo numérico: aspectos teóricos*

*e computacionais*. Pearson/Makron, 1998.

““

# Apêndice B

## Propostas para Novos Projetos

---

Propostas para projetos que podem ser gerados como continuação deste projeto. Apenas texto simples pode ser utilizado para responder a estes campos.

### B.1 Proposta 1

#### B.1.1 Resumo

Título:

Palavras-chave:

Resumo:

#### B.1.2 Introdução/Justificativa

*Incluindo os benefícios esperados no processo ensino-aprendizagem e o retorno para os cursos e para os professores do CEFET-MG em geral*

#### B.1.3 Objetivos

Objetivo geral

Objetivos específicos

#### B.1.4 Metodologia

#### B.1.5 Referências

#### B.1.6 Cronograma